NHÓM 6

BÁO CÁO ĐỒ ÁN THUẬT TOÁN A *

GVHD: Ts. Nguyễn An Tế

Nguyễn Phan Nhật Lan

Kim Phan Thúy Liễu

Vương Thùy Linh

Nguyễn Đỗ Nhật Minh

Trần Khánh Ngân

TỔNG QUAN A*

Lịch sử hình thành:

- Phát triển năm 1968 bởi Hart, Nilsson, Raphael.
- Tại Viện Nghiên cứu Stanford (SRI International).

Thuật toán A*:

- Là thuật toán tìm kiếm đường đi tốt nhất, được mở rộng từ thuật toán Dijkstra.
- **Phương pháp:** sử dụng hàm đánh giá f(n)
- **Ưu điểm:** nhanh hơn, hiệu quả hơn với độ phức tạp thường nhỏ hơn so với Dijkstra.
- **Đảm bảo:** tối ưu với heuristic tốt

Đặc trưng của thuật toán A*:

- Sử dụng hàm đánh giá f(n): ước lượng chi phí từ n đến đích.
- Giúp hướng dẫn tìm kiếm thông minh, giảm thời gian.

$$f(n) = g(n) + h(n)$$

g(n): chi phí thực tế từ điểm bắt đầu đến nút n
h(n): Ước tính chi phí từ nút *n* đến đích.
f(n): Tổng chi phí ước tính của đường đi qua *n*.

MÔ TẢ BÀI TOÁN

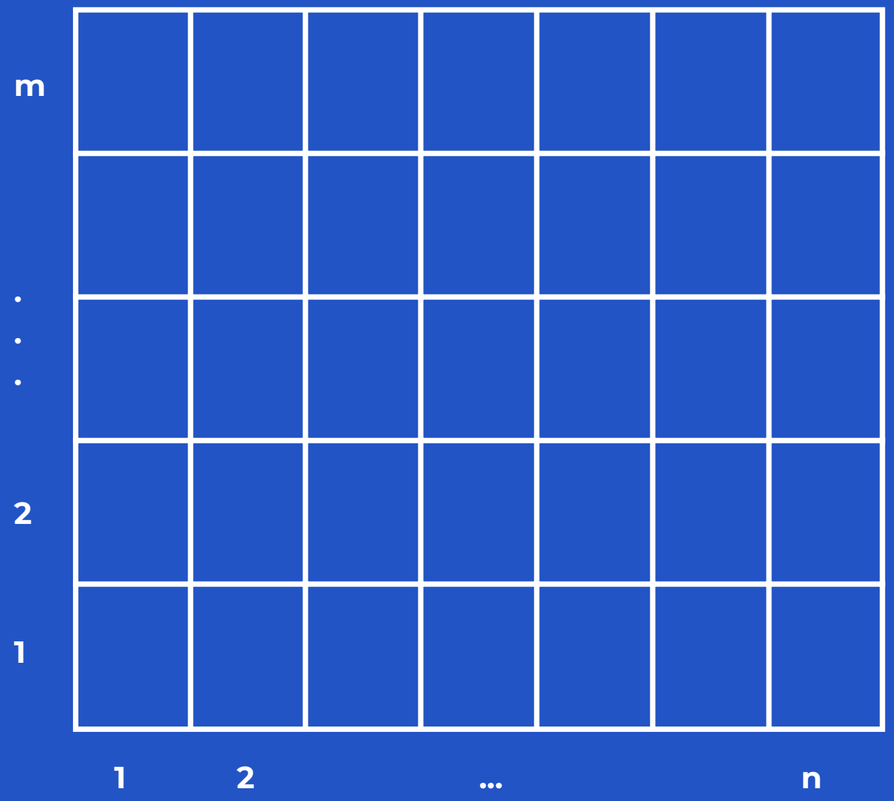
Mô tả bài toán: Cho ma trận 2 chiều A_{mxn} với m, n thuộc R như hình

Một robot hút bụi sẽ được đặt vào vị trí bất kì của ma trận và có thể thực hiện một trong hai loại hành động:

- (M - move): di chuyển sang 1 trong 8 hướng xung quanh ô đang đứng
- (S - suck): làm sạch ô đang đứng

Yêu cầu: viết chương trình để robot thực hiện việc hút bụi với các điều kiện:

1. Người dùng chỉ định kích thước ma trận
2. Người sử dụng chỉ định số lượng (D) bất kì và chương trình sẽ sinh ra các ô (D) ở vị trí ngẫu nhiên trong ma trận.
3. Sử dụng thuật toán A* để tìm ra đường đi phù hợp cho robot hút bụi
4. Thể hiện kết quả bài toán: lộ trình của robot và tổng chi phí



MỤC TIÊU BÀI TOÁN

MỤC TIÊU TỔNG QUÁT

Tìm chuỗi hành động (Move, Suck) của robot sao cho dọn sạch mọi ô bẩn với tổng chi phí nhỏ nhất.

QUY TẮC CHI PHÍ

Tần suất hành động

Robot thực hiện 1 hành động mỗi lượt.

Chi phí di chuyển

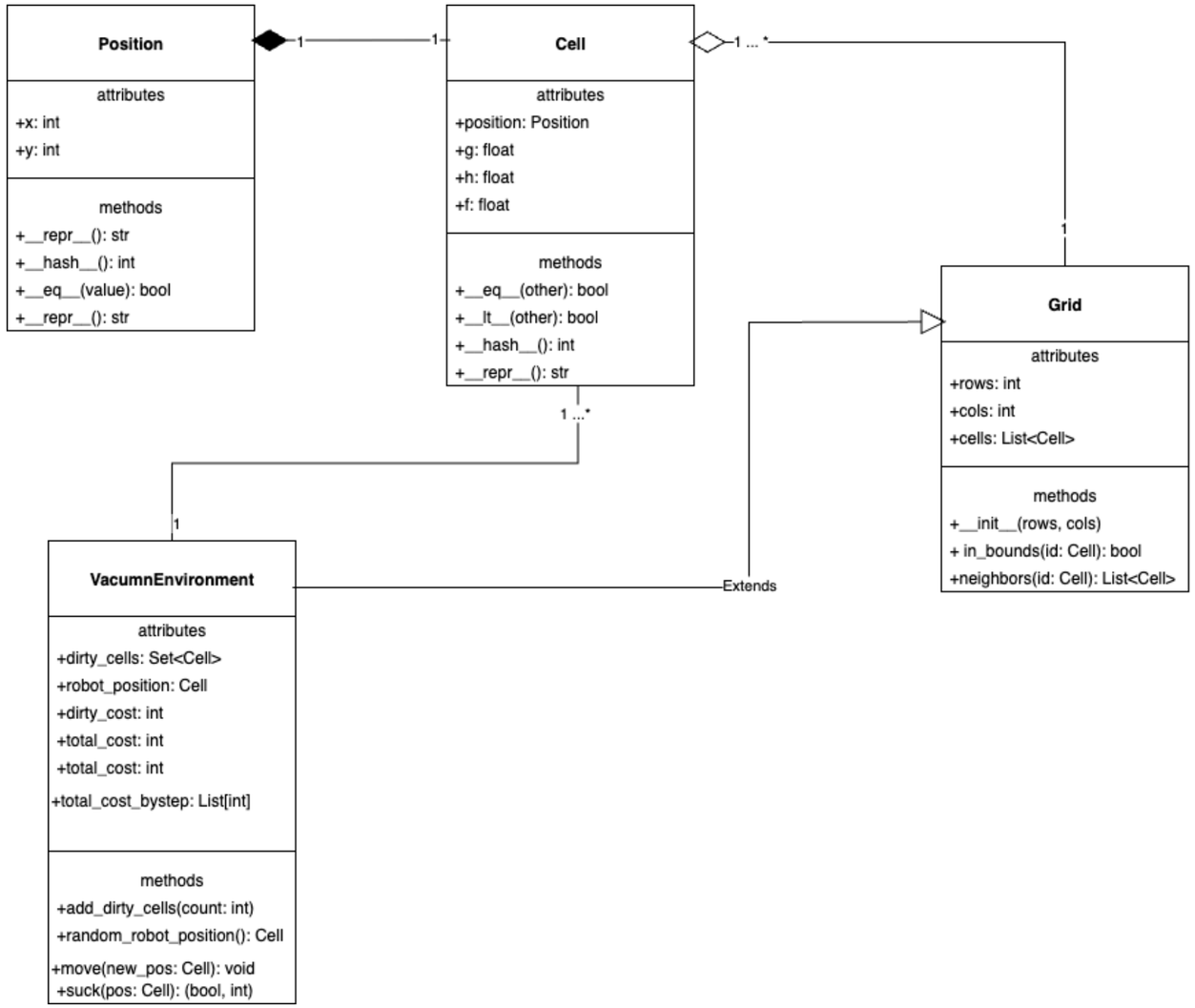
1 đơn vị cho mỗi hành động di chuyển



Chi phí hút bụi (Suck)

Tăng 1 đơn vị sau mỗi lượt hành động với mỗi ô bẩn (D) còn lại.

Thiết kế class cho bài toán



Lớp	Mô tả ngắn gọn
Position	Quản lý tọa độ (x, y) trên lưới
Cell	Lưu vị trí + giá trị g, h, f của A*
Grid	Lưới 2D chứa các Cell, cung cấp neighbors()
VacuumEnvironment	Kế thừa Grid, thêm robot, tập ô bẩn, chi phí hành động

Môi trường bài toán

VacumnEnvironment kế thừa từ Grid → tạo lưới A(m, n).

Thành phần chính:

- **robot_position** – vị trí robot
- **dirty_cells** – tập ô bẩn (Set).
- **dirty_cost** = 1, **total_cost** = 0,
- **total_cost_bystep** – chi phí tích luỹ

Hành động:

- **add_dirty_cells()** – sinh ngẫu nhiên ô bẩn.
- **random_robot_position()** – chọn vị trí robot.
- **move()** – di chuyển, tăng chi phí tổng + tăng chi phí làm sạch.
- **suck()** – hút bụi, tăng chi phí tổng + tăng chi phí làm sạch.

```python
class VacuumEnvironment(Grid):
    def __init__(self, rows: int, cols: int):
        super().__init__(rows, cols)
        self.dirty_cells = set()
        self.robot_position = Cell(Position(0,0))
        self.dirty_cost = 1
        self.total_cost = 0
        self.total_cost_bystep = []

    def add_dirty_cells(self, count: int):
        while len(self.dirty_cells) < count:
            x = random.randint(0, self.cols - 1)
            y = random.randint(0, self.rows - 1)
            self.dirty_cells.add(Cell(Position(x, y)))

    def random_robot_position(self):
        r = random.randint(0, self.rows*self.cols-1)
        return self.cells[r]

    def suck(self, pos):
        if pos in self.dirty_cells:
            self.dirty_cells.remove(pos)

            suck_cost = self.dirty_cost
            self.total_cost += suck_cost
            self.dirty_cost+=1
            return True, suck_cost
        return False, 0

    def move(self, new_pos):

        self.total_cost += 1
        self.dirty_cost += 1
        self.robot_position = new_pos
```

THUẬT TOÁN A*

Yêu cầu hàm heuristic h(n):

Admissible: Không được ước lượng chi phí cao hơn chi phí thực tế.

Ý tưởng

Kết hợp Greedy BFS và Dijkstra để tạo ra thế cân bằng cho việc tìm kiếm đường đi giữa hai thuật toán

- ***frontier***: Hàng đợi ưu tiên (Priority Queue). Chi phí (f = g+h) đóng vai trò xếp hạng thứ tự xếp hạng được lấy ra từ hàng đợi
- ***came_from***: Dictionary với Key là Cell hiện tại, Value là Cell Parent
- ***cost_so_far***: Dictionary với Key là Cell hiện tại, Value là chi phí di chuyển tính từ điểm bắt đầu
- ***heuristic(goal, next)***: Là hàm tính khoảng cách nhỏ nhất từ điểm từ Cell tiếp theo đến Goal
- ***new_cost***: Chi phí g tính từ Start đến Cell tiếp theo

```python
def a_star_multigoal(graph, start, goals):
    frontier = PriorityQueue()
    frontier.put((0, start))
    came_from = {}
    cost_so_far = {}
    came_from[start] = None
    cost_so_far[start] = 0

    while not frontier.empty():
        priority_cost, current = frontier.get()


        if current in goals:
            break

        for next in graph.neighbors(current):
            new_cost = cost_so_far[current] + 1
            if next not in cost_so_far or new_cost < cost_so_far[next]:
                cost_so_far[next] = new_cost
                priority = new_cost + multi_goal_heuristic(next, goals)
                frontier.put((priority,next))
                came_from[next] = current

    return came_from, cost_so_far, current
```

Hàm heuristic

```python
def heuristic(a: Cell, b: Cell):
    return max(abs(a.position.x - b.position.x), abs(a.position.y - b.position.y))


def multi_goal_heuristic(current: Cell, goals: Cell):
    if not goals:
        return 0
    return min(heuristic(current, goal) for goal in goals)
```

- Heuristic đơn đích:
  h=Chebyshev distance đến 1 ô bẩn
- Heuristic đa đích:
  h=min(Chebyshev distance đến các ô bẩn còn lại)

→ Cho phép A* ưu tiên ô bẩn gần nhất, giúp giảm không gian tìm kiếm.

Chiến lược hoạt động của Robot

```
env = VacuumEnvironment(rows,cols)
env.robot_position = robot_start
env.dirty_cells = set(dirty_positions)

initial_state = {
    'robot_pos': robot_start,
    'dirty_cells': dirty_positions.copy()
}

path, total_cost, cost_by_step = vacuum_cleaning_plan(env)
```

Input: (VacuumEnvironment)

gồm:

robot_position – vị trí khởi tạo robot

dirty_cells – tập ô bẩn ban đầu

dirty_cost, total_cost,

total_cost_bystep – các biến theo dõi chi phí

Output:

path – chuỗi hành động

total_cost – tổng chi phí sau khi hoàn tất

total_cost_bystep – giá trị total_cost sau mỗi hành động

Chiến lược hoạt động

Ý tưởng: Lặp lại A* nhiều lần, mỗi lần tìm đến ô bẩn gần nhất

Nếu robot đang ở ô bẩn → suck(). Nếu không:

Dùng A* để tìm đường tới ô bẩn gần nhất.

Di chuyển theo đường đó, cập nhật chi phí.

Hút ô bẩn khi đến nơi.

Lặp lại cho tới khi không còn ô bẩn.

```python
def vacuum_cleaning_plan(environment: VacuumEnvironment):
    path = []
    current_pos = environment.robot_position


    if current_pos in environment.dirty_cells:
        success, cost = environment.suck(current_pos)
        if success:
            path.append(('suck', current_pos, cost))
            environment.total_cost_bystep.append(environment.total_cost)


    while environment.dirty_cells:
        goals = list(environment.dirty_cells)
        came_from, cost_so_far, reached_goal = a_star_multigoal(
            environment, current_pos, goals)


        partial_path = reconstruct_path(came_from, current_pos, reached_goal)

        for pos in partial_path[1:]:
            environment.move(pos)
            path.append(('move', pos))

            environment.total_cost_bystep.append(environment.total_cost)

        success, cost = environment.suck(reached_goal)
        if success:
            path.append(('suck', reached_goal, cost))
            environment.total_cost_bystep.append(environment.total_cost)


        current_pos = reached_goal

    return path, environment.total_cost, environment.total_cost_bystep
```

CHƯƠNG TRÌNH NHẬP

Bước 1: Nhập kích thước lưới & số ô bẩn

Bước 2: Khởi tạo môi trường bài toán

Tạo VacuumEnvironment(rows, cols)

Bước 3: Nhập tọa độ robot (gốc trái-dưới, bắt đầu từ 1)

Bước 4: Chuẩn hóa tọa độ về hệ máy (gốc trái-trên)

Bước 5: Gọi các hàm chính
get_cleaning_steps() → sinh danh sách hành động
print_steps() → in ra các bước.

Bước 6: Hiển thị Visualization (nếu có)

```
while True:
    try:
        cols = int(input("Nhập số cột: "))
        rows = int(input("Nhập số dòng: "))
        if 1 <= cols and 1 <= rows:
            break
        print("Kích thước phải là số nguyên từ 1 trở lên")
    except ValueError:
        print("Vui lòng nhập số nguyên!")

max_dirty  = cols*rows

while True:
    try:
        dirty_count = int(input(f"Nhập số lượng ô dirty (1-{max_dirty}): "))
        if 1 <= dirty_count <= max_dirty:
            break
    except ValueError:
        print("Vui lòng nhập số nguyên!")

env = VacuumEnvironment(rows, cols)
env.add_dirty_cells(dirty_count)

while True:
    random_start = input("Random vị trí xuất phát của robot (Y/N): ").strip().upper()
    if random_start == 'Y':
        robot_start = env.random_robot_position()
        break
    else:
        start_x = int(input(f"Nhập vị trí x của robot (1-{cols}): "))
        start_y = int(input(f"Nhập vị trí y của robot (1-{rows}): "))
        sys_x = start_x - 1
        sys_y = rows - start_y
        robot_start = Cell(Position(sys_x, sys_y))

        if env.in_bounds(robot_start):
            break
        print(f"Tọa độ robot phải trong khoảng: x(1-{cols}), y(1-{rows})")

dirty_positions = list(env.dirty_cells)
result = get_cleaning_steps(rows, cols, robot_start, dirty_positions)
```

UI

UI gồm 2 lớp:
VisualGrid: vẽ lưới, robot, đường đi.
GridAnimate: điều khiển và trình chiếu bước chạy.

Tương tác: Slider điều khiển từng bước → canvas cập nhật khung hình trực quan.

```
class VisualGrid:
    def __init__(self, rows, cols, cell_size= 50):…

    def display(self):…

    def _fillCell(self, x, y, color, alpha=1):…

    def _fillStroke(self, x, y, color='#444', cols=2):…

    def _cellText(self, x, y, text, color=None, size=16):…


    def drawRobot(self, r, c):…

    def drawPath(self, path):…

    def drawGrid(self):…
```

✓ 0.0s

```
class GridAnimate:
    def __init__(self, grid, steps):…

    def playComponent(self):…

    def widgetComponent(self, n):…

    def Displaying(self):…

    def renderStep(self, step, show_scores=False):…
```

✓ 0.0s

reconstruct_path(): dùng để truy vết đường đi
Kiểm tra xem goal có trong came_from.
Nếu có, lần ngược từ goal về start qua các nút cha.
Thêm start vào danh sách, đảo ngược để được đường đi đúng hướng.

Kết quả:
Trả về danh sách các nút (Cells hoặc Positions) theo thứ tự từ start → goal.
Nếu không tìm thấy đường đi → trả về [].

CÁC HÀM HỖ TRỢ

```
def reconstruct_path(came_from, start, goal):
    current = goal
    path = []
    if goal not in came_from:
        return []
    while current != start:
        path.append(current)
        current = came_from[current]
    path.append(start)
    path.reverse()
    return path
```

get_cleaning_step()

Cách hoạt động:

Tạo môi trường VacuumEnvironment(rows, cols).

Gán vị trí robot và tập các ô bẩn.

Gọi thuật toán vacuum_cleaning_plan(env) để tìm đường đi tối ưu.

Tính số bước MOVE và SUCK.

Trả về kết quả gồm: đường đi, chi phí, vị trí cuối, và thống kê.

.

```
def get_cleaning_steps(rows: int, cols: int, robot_start: Cell,
                       dirty_positions):

    env = VacuumEnvironment(rows,cols)
    env.robot_position = robot_start
    env.dirty_cells = set(dirty_positions)

    initial_state = {
        'robot_pos': robot_start,
        'dirty_cells': dirty_positions.copy()
    }

    path, total_cost, cost_by_step = vacuum_cleaning_plan(env)
    move_count = sum(1 for a in path if a[0] == 'move')
    suck_count = sum(1 for a in path if a[0] == 'suck')

    final_position = env.robot_position

    return {
        'steps': path,
        'total_cost': total_cost,
        'cost_by_step': cost_by_step,
        'move_count': move_count,
        'suck_count': suck_count,
        'initial_state': initial_state,
        'final_position': final_position,
        'rows': rows
    }
```

```python
def print_steps(result):
    rows = result['rows']

    print("KẾT QUẢ ROBOT HÚT BỤI")

    print(f"\nTrạng thái ban đầu:")
    robot_pos = result['initial_state']['robot_pos']
    x, y = robot_pos.position.x, robot_pos.position.y

    user_x = x + 1
    user_y = rows - y
    print(f"- Robot tại: ({user_x}, {user_y})")

    dirty_cells = result['initial_state']['dirty_cells']
    print(f"- Các ô bẩn: {[(cell.position.x + 1, rows - cell.position.y) for cell in dirty_cells ]}")

    print(f"\nLộ trình chi tiết ({len(result['steps'])} bước - Đã chuẩn hoá về toạ độ người dùng):")
    for i, action in enumerate(result['steps'], 1):
        x, y = action[1].position.x, action[1].position.y
        user_x = x + 1
        user_y = rows - y
        if action[0] == 'move':
            print(f"  Bước {i}: MOVE → ({user_x}, {user_y})| Chi phí tổng: {result['cost_by_step'][i-1]}")
        elif action[0] == 'suck':
            print(f"  Bước {i}: SUCK tại ({user_x}, {user_y}) | Chi phí hút ô dơ: {action[2]} | Chi phí tổng: {

    print(f"\nThống kê:")
    print(f"- Tổng chi phí: {result['total_cost']}")
    print(f"- Số lần MOVE: {result['move_count']}")
    print(f"- Số lần SUCK: {result['suck_count']}")

    res = result['final_position']
    x, y = res.position.x , res.position.y
    print(f"- Vị trí cuối: ({x + 1}, {rows - y})")

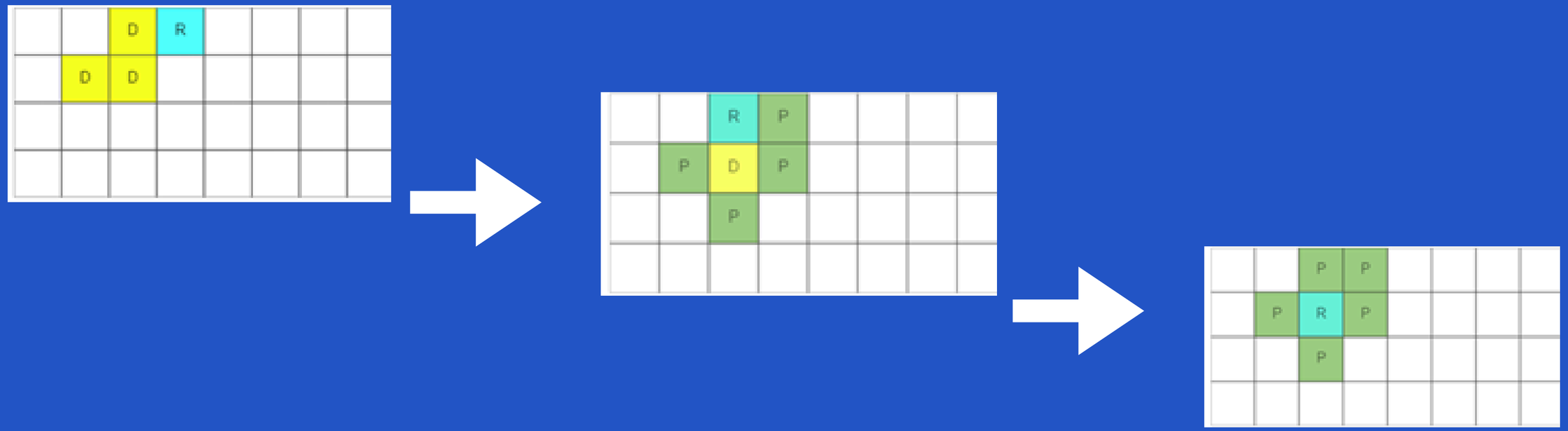
    if result['suck_count'] > 0:
        avg_cost = result['total_cost'] / result['suck_count']
        print(f"  - Chi phí trung bình/ô: {avg_cost:.2f}")
```

Hàm print_steps()

Hỗ trợ việc in và thống kê kết qủa

Kết quả và phân tích

Mục tiêu: Kiểm chứng hoạt động của thuật toán A* trên các bản đồ có kích thước và số lượng ô bẩn khác nhau, qua đó đánh giá khả năng tìm đường tối ưu.



THỰC NGHIỆM

Case	Kích thước lưới	Vị trí robot	Số lượng ô bẩn	Tổng chi phí	Số lần di chuyển (MOVE)	Số lần làm sạch (SUCK)	Vị trí cuối	Chi phí trung bình/ô
1	5x5	Random ngẫu nhiên	2	9	3	2	(1,4)	4.50
2	8x4	Nhập thủ công	3	23	5	3	(3,3)	7.61
3	7x1	Random ngẫu nhiên	5	38	7	5	(2,1)	7.60

Kết quả

Case	Tổng chi phí	Số lần di chuyển (MOVE)	Số lần làm sạch (SUCK)	Vị trí cuối
1	9	3	2	(1,4)
2	23	5	3	(3,3)
3	38	7	5	(2,1)

Case 3

BẨN*: (6, 1), (5, 1), (3, 1), (2, 1), (7, 1)

STEP



Trạng thái ban đầu: Robot tại (5, 1)

1



2



3



4



5



6



7



8



9



10



11



12



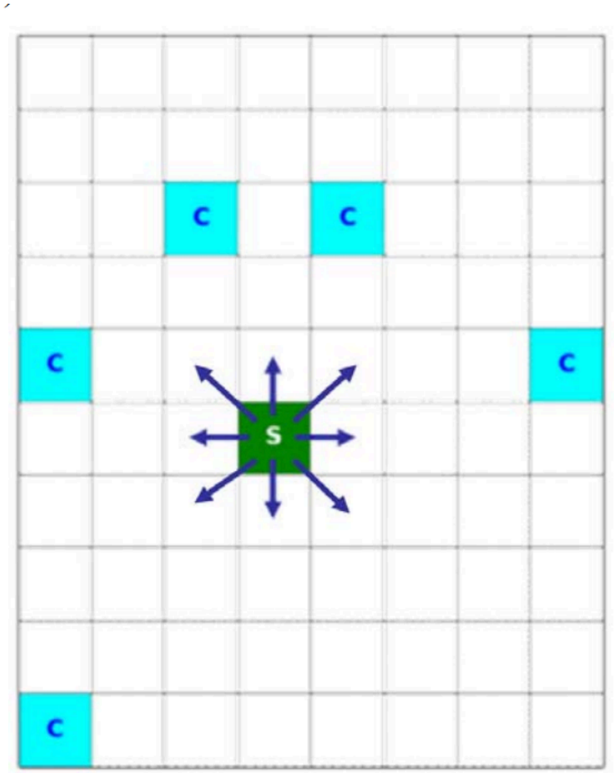
- Bước 1: SUCK tại (5, 1), chi phí làm sạch là 1 và chi phí tổng là 1
- Bước 2: MOVE → (6, 1), chi phí tổng: 2
- Bước 3: SUCK tại (6, 1), chi phí làm sạch là 3 và chi phí tổng: 5
- Bước 4: MOVE → (7, 1), Chi phí tổng: 6
- Bước 5: SUCK tại (7, 1), chi phí làm sạch là 5 và chi phí tổng là 11
- Bước 6: MOVE → (6, 1), chi phí tổng: 12
- Bước 7: MOVE → (5, 1), chi phí tổng: 13
- Bước 8: MOVE → (4, 1), chi phí tổng: 14
- Bước 9: MOVE → (3, 1), chi phí tổng: 15
- Bước 10: SUCK tại (3, 1) , Chi phí hút ô dơ: 10 , Chi phí tổng: 25
- Bước 11: MOVE → (2, 1), Chi phí tổng: 26
- Bước 12: SUCK tại (2, 1) , Chi phí hút ô dơ: 12 , Chi phí tổng: 38

HẠN CHẾ*



Bước chạy thứ 3 → (1,3) TỔNG CHI PHÍ : 35

Nếu → (1,1) TỔNG CHI PHÍ 33 → TỐI ƯU HƠN



*Nhận xét:

- Robot hiện tại dọn theo chiến lược nearest-first (A* để tới một mục tiêu rồi suck), nhưng kết quả không tối ưu về tổng chi phí
- Nó chỉ tối ưu cho bước hiện tại
- Chưa cân nhắc được chi phí tương lai - Chi phí “Suck” tăng theo số hành động

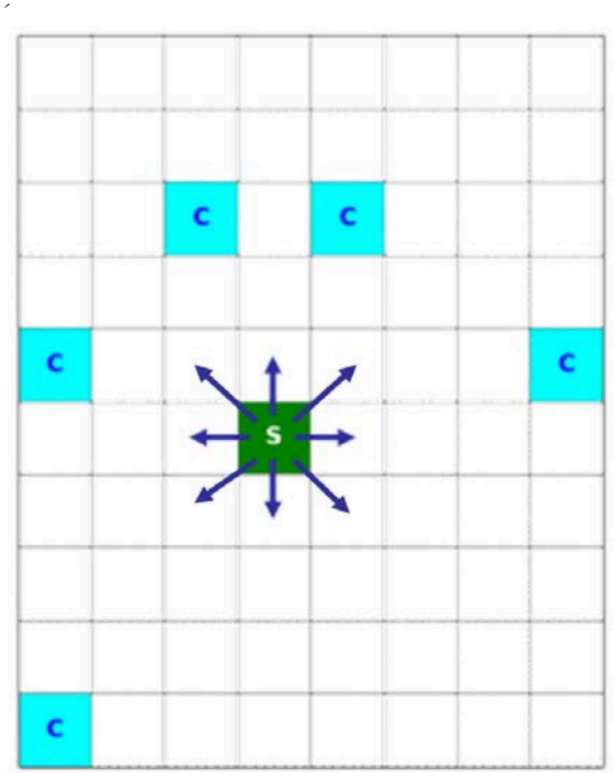
MỞ RỘNG

A* trên không gian trạng thái mở rộng

- Có thể biểu diễn toàn bộ trạng thái gồm vị trí hiện tại và các ô bẩn còn lại

- Khi đó, A* sẽ tìm chuỗi hành động tối ưu toàn cục, xét mọi khả năng để giảm tổng chi phí dài hạn.

- Lưu ý: cách này đòi hỏi không gian tìm kiếm rất lớn và tính toán nặng hơn, nên thường chỉ dùng khi bản đồ nhỏ hoặc chi phí “sai đường” quá đắt.



KẾT LUẬN

Hạn chế: Tổng chi phí dọn dẹp chưa tối ưu trong 1 số trường hợp

KẾT QUẢ

1. Xây dựng thành công hàm Heuristics đơn giản và Multi-goals A* algorithm

2. Giải quyết được bài toán Robot dọn dẹp trên lưới

3. Đảm bảo: Admissible Heuristic, chi phí “Suck” tăng theo số hành động, tất cả ô “Dirty” được làm sạch và các yêu cầu khác của bài toán

4. Xây dựng UI đơn giản → Trực quan hóa

NHÓM 6

BÁO CÁO ĐỒ ÁN THUẬT TOÁN DIJKSTRA

GVHD: TS. Nguyễn An Tế

TỔNG QUAN DIJKSTRA

Lịch sử hình thành:

- Nghĩ ra và phát triển vào năm 1956 bởi Edsger W. Dijkstra

Thuật toán Dijkstra:

- Là thuật toán tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm

Đặc trưng:

- Khi một đỉnh được chọn, đường đi ngắn nhất đến đỉnh đó đã được xác định.
- Nhờ chiến lược chọn tham lam theo khoảng cách tạm thời nhỏ nhất.
- Điều kiện bắt buộc: trọng số các cạnh ≥ 0.

Độ phức tạp

- Thời gian: O((V + E) log V)
- Không gian: O(V)

MÔ TẢ BÀI TOÁN

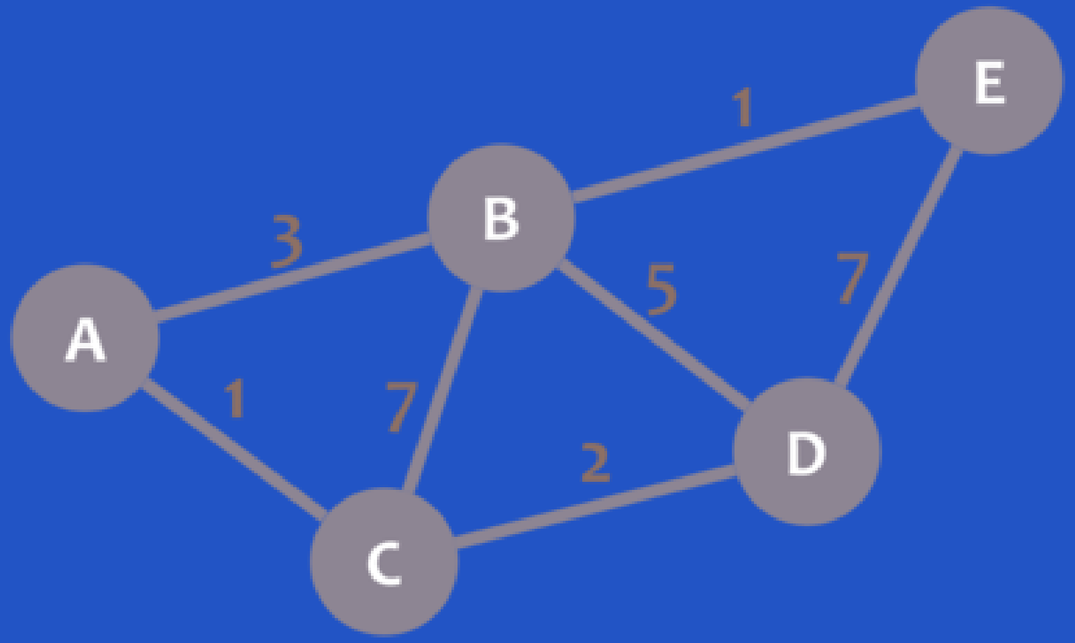
Bài toán:
Tìm đường đi ngắn nhất giữa hai đỉnh bất kỳ trong đồ thị có trọng số không âm.

Dữ liệu:
Đồ thị vô hướng G được lưu trong file Graph.csv, mỗi dòng chứa:

- (v_from, v_to, weight)

Yêu cầu:

- Cài đặt thuật toán Dijkstra bằng Python (hướng đối tượng, file .ipynb).
- Hiển thị đường đi ngắn nhất và độ dài tương ứng.



THUẬT TOÁN DIJKSTRA

Đặc tả hình thức:

- **pq**: Hàng đợi ưu tiên (min-heap) lưu trữ cặp (khoảng cách tạm thời, đỉnh) - dùng để chọn đỉnh có chi phí nhỏ nhất được mở rộng tiếp theo.

- **dist**: Dictionary với Key = đỉnh, Value = chi phí nhỏ nhất hiện tại từ điểm bắt đầu đến đỉnh đó.

- **prev**: Dictionary với Key = đỉnh hiện tại, Value = đỉnh cha - dùng để truy vết đường đi ngắn nhất sau khi thuật toán kết thúc.

- **new_dist**: Chi phí mới khi đi từ đỉnh hiện tại u đến đỉnh kề v:

=> new_dist = current_dist + weight(u, v).

- **Điều kiện cập nhật**: Nếu new_dist < dist[v] thì cập nhật lại dist[v] = new_dist, prev[v] = u và thêm (new_dist, v) vào pq.

THIẾT KẾ CLASS

Lớp	Chức năng chính	Mô tả
Node	Đại diện cho một đỉnh	Lưu trữ tên đỉnh, khoảng cách tạm thời, danh sách cạnh kề, và trạng thái
Edge	Đại diện cho một cạnh giữa hai đỉnh	Lưu trữ đỉnh đầu (v_from), đỉnh cuối (v_to) và trọng số (weight)
Graph	Quản lý toàn bộ đồ thị	Chứa danh sách các đỉnh và cạnh, cung cấp hàm thêm đỉnh/cạnh, đọc dữ liệu từ file và tạo danh sách kề
Dijkstra	Thực thi thuật toán Dijkstra	Cài đặt thuật toán tìm đường đi ngắn nhất, cập nhật khoảng cách, chọn đỉnh nhỏ nhất, và xuất kết quả đường đi + chi phí
Animate	Quản lý hiệu ứng chuyển động	Định nghĩa trạng thái và bước chuyển động cho từng bước
AnimateController	Điều khiển quá trình chuyển động	Điều phối, cập nhật trạng thái animation theo thời gian, xử lý tương tác và điều khiển tiến trình

Cấu trúc Đỉnh và Cạnh đồ thị

Lớp Node

- Đại diện cho đỉnh trong đồ thị.
- **Thuộc tính:**
  - name: tên hoặc định danh của đỉnh.
- **Phương thức quan trọng:**
  - __eq__ và __hash__: so sánh hai đối tượng giống/ khác nhau, đồng thời cho phép Node làm khóa trong dictionary.

Lớp Edge

- Đại diện cho cạnh giữa 2 đỉnh trong đồ thị có trọng số.
- **Thuộc tính:**
  - v_from: đỉnh đầu.
  - v_to: đỉnh cuối.
  - weight: trọng số của cạnh.
- **Phương thức quan trọng:**
  - other(node): trả về đỉnh đối diện với node trên cạnh

```python
class Node:
  def __init__(self, name):
    self.name = name
  def __eq__(self, other):
    return self.name == other.name
  def __hash__(self):
    return hash(self.name)
  def __repr__(self):
    return f"Node({self.name})"


class Edge:
  def __init__(self, v_from, v_to, weight):
    self.v_from = v_from
    self.v_to = v_to
    self.weight = weight
  def other(self, node):
    if node == self.v_from:
        return self.v_to
    elif node == self.v_to:
        return self.v_from
    else:
        raise ValueError(f"{node} không thuộc cạnh này")
  def __repr__(self):
    return f"Edge({self.v_from.name} - {self.v_to.name}, W={self.weight})"
```

Biểu diễn đồ thị vô hướng có trọng số

Lớp Graph

- **Thuộc tính:**
  - **adj**: dictionary lưu trữ các đỉnh kề và trọng số cạnh.
  - **nodes:** dictionary các đối tượng Node.
  - **edges:** danh sách các cạnh (Edge) trong đồ thị.

- **Phương thức chính:**
  - **add_edge(u_name, v_name, weight)**: Thêm cạnh giữa hai đỉnh, cập nhật danh sách kề 2 chiều.
  - **load_from_csv(filename):** Đọc dữ liệu đồ thị từ file CSV theo định dạng (v_from, v_to, weight).
  - **get_neighbors(node_name):** Lấy danh sách các đỉnh kề cùng trọng số cạnh.
  - **vertices():** Trả về danh sách các đỉnh của đồ thị.

```
class Graph:
  def __init__(self):
    self.adj = {}
    self.nodes = {}
    self.edges = []
  def add_edge(self, u_name, v_name, weight):
    u = self.nodes.setdefault(u_name, Node(u_name))
    v = self.nodes.setdefault(v_name, Node(v_name))
    edge = Edge(u, v, weight)
    self.edges.append(edge)


    if u_name not in self.adj:
      self.adj[u_name] = []
    if v_name not in self.adj:
      self.adj[v_name] = []


    if (v_name, weight) not in self.adj[u_name]:
      self.adj[u_name].append((v_name, weight))
    if (u_name, weight) not in self.adj[v_name]:
      self.adj[v_name].append((u_name, weight))

  def load_from_csv(self, filename):
    with open(filename, 'r', encoding='utf-8') as f:
      reader = csv.DictReader(f)
      for row in reader:
        u = row['v_from'].strip()
        v = row['v_to'].strip()
        w = float(row['weight'])
        self.add_edge(u, v, w)

  def get_neighbors(self, node_name):
    return self.adj.get(node_name, [])

  def vertices(self):
    return list(self.adj.keys())
```

Lớp Dijkstra

Hàm shortest_path

Ý tưởng chính

- Sử dụng priority queue để chọn đỉnh có khoảng cách nhỏ nhất chưa xử lý.
- Cập nhật khoảng cách đến các đỉnh lân cận nếu tìm được đường đi ngắn hơn.
- Ghi lại các bước xử lý để hỗ trợ minh họa thuật toán.

Cách hoạt động

- Khởi tạo:

- dist: khoảng cách đến tất cả đỉnh = vô cực, trừ đỉnh bắt đầu = 0.
- prev: lưu đỉnh trước đó trên đường đi ngắn nhất.
- pq: hàng đợi ưu tiên với (khoảng cách, đỉnh).
- settled: tập các đỉnh đã được xử lý.

```
def shortest_path(self, start):
    self.steps = []
    dist = {v: float('inf') for v in self.graph.vertices()}
    dist[start] = 0
    prev = {v: None for v in self.graph.vertices()}

    pq = [(0, start)]
    settled = set()

    while pq:
        current_dist, u = heapq.heappop(pq)
        if u in settled:
            continue
        settled.add(u)

        self.steps.append({
            "t": len(self.steps) + 1,
            "type": "settle",
            "current": u,
            "settled": list(settled),
            "dist": dict(dist),
            "path": self.get_path(prev, start, u)
        })
        for v, weight in self.graph.get_neighbors(u):
            if v in settled:
                continue
            new_dist = current_dist + weight
            updated = False
            if new_dist < dist[v]:
                dist[v] = new_dist
                prev[v] = u
                heapq.heappush(pq, (new_dist, v))
                updated = True

            self.steps.append({
                "t": len(self.steps) + 1,
                "type": "edge_check",
                "current": u,
                "neighbor": v,
                "weight": weight,
                "new_dist": new_dist,
                "updated": updated,
                "settled": list(settled),
                "dist": dict(dist),
                "path": self.get_path(prev, start, u)
            })
    return dist, prev
```

Lớp Dijkstra

Hàm get_path

Chức năng:

- Dựng lại đường đi từ start đến end dựa trên mảng prev.

Cách hoạt động:

1. Bắt đầu từ đỉnh end.
2. Lần theo prev[node] ngược lại cho đến khi về start.
3. Đảo ngược danh sách để ra thứ tự đúng.

Kết quả:

- Trả về danh sách các đỉnh trên đường đi.

```python
def get_path(self, prev, start, end):
    path = []
    node = end
    while node is not None:
        path.insert(0, node)
        node = prev[node]
    if len(path) == 0 or path[0] != start:
        return None
    return path
```

Hàm find_shortest_path

Chức năng: Gọi thuật toán Dijkstra để lấy:

- dist: khoảng cách ngắn nhất từ start đến mọi đỉnh
- prev: đỉnh liền trước trên đường đi
- Dựng lại đường đi bằng get_path.

Kết quả:

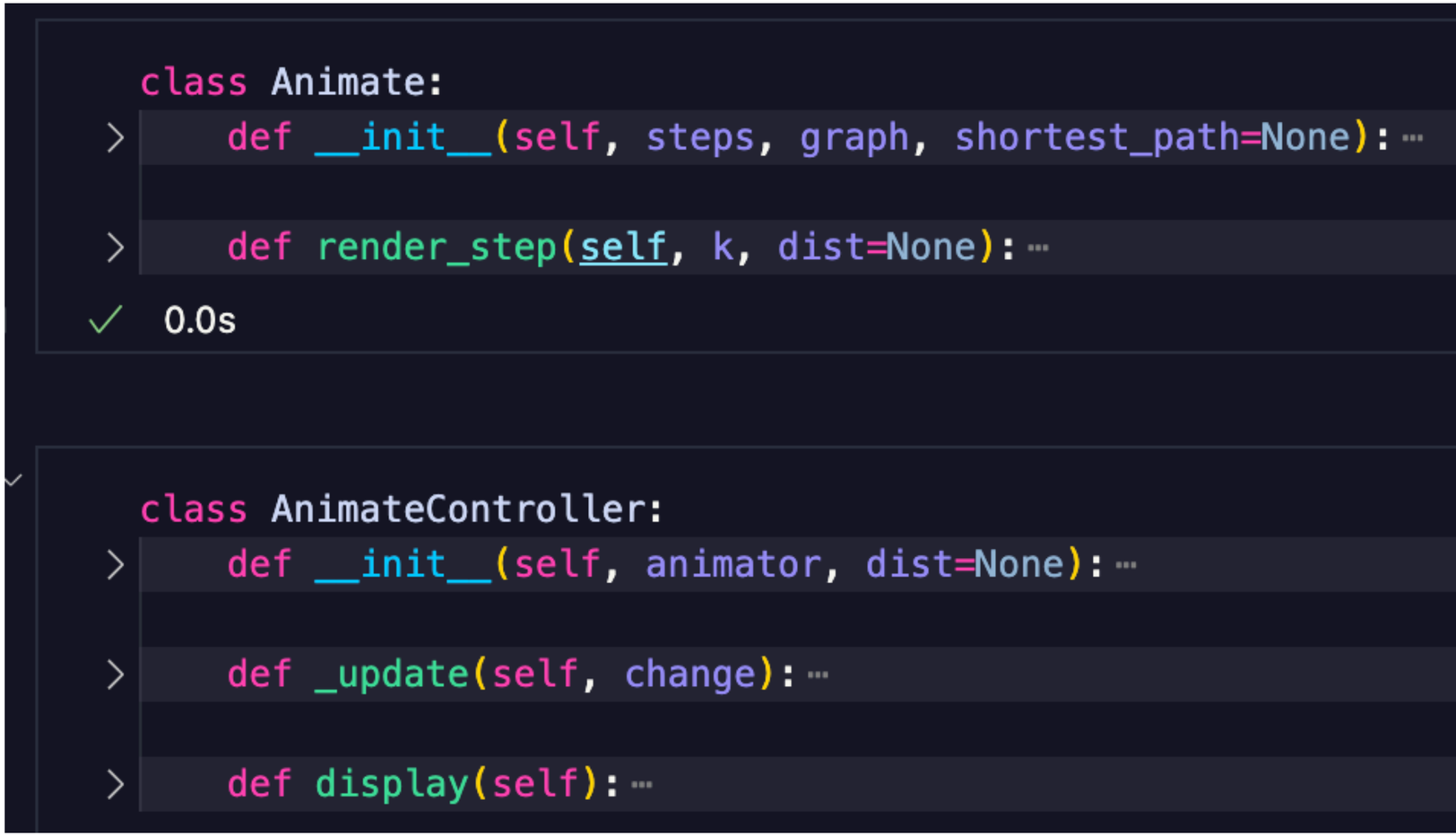
- (khoảng cách ngắn nhất, danh sách đường đi)

```python
def find_shortest_path(self, start, end):
    dist, prev = self.shortest_path(start)
    path = self.get_path(prev, start, end)
    return dist[end], path
```

TRỰC QUAN HOÁ THUẬT TOÁN

Class Animate: Khởi tạo đồ thị để trực quan, tô màu các đỉnh và vẽ cạnh

- Đỉnh hiện tại: cam
- Đỉnh đã duyệt: xanh lá
- Đỉnh biên: vàng
- Khác: xám
- Đường đi ngắn nhất: xanh tím
- Cạnh đang xét: màu đỏ.
- Đường đi ngắn nhất: màu tím.



Class AnimateController:

Cho phép render từng step qua hàm _update()

KẾT QUẢ THỰC NGHIỆM - TH1

CẤU TRÚC FILE CSV

A,B,1

A,C,2

B,D,10

B,E,2

C,E,3

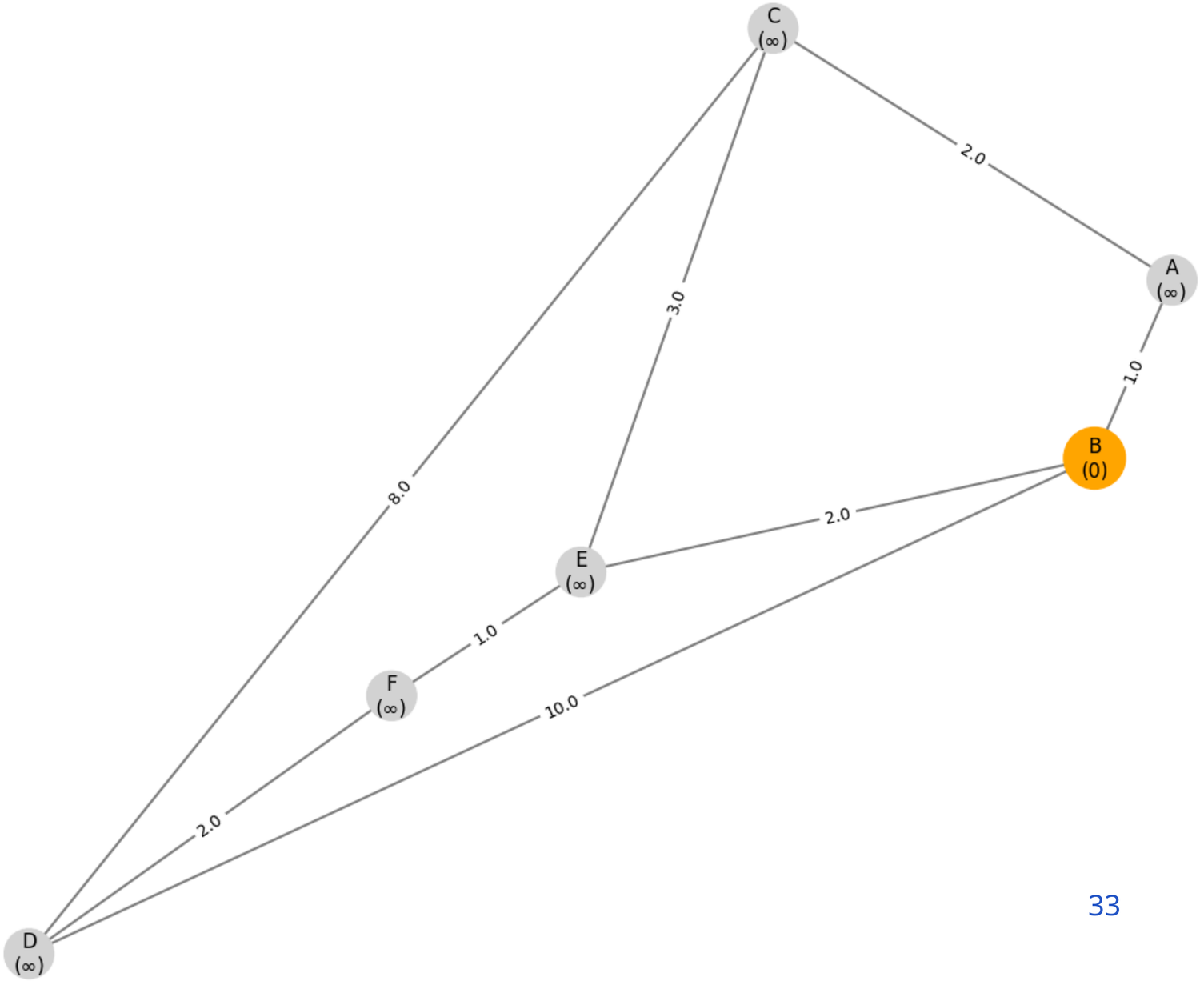
C,D,8

E,F,1

D,F,2

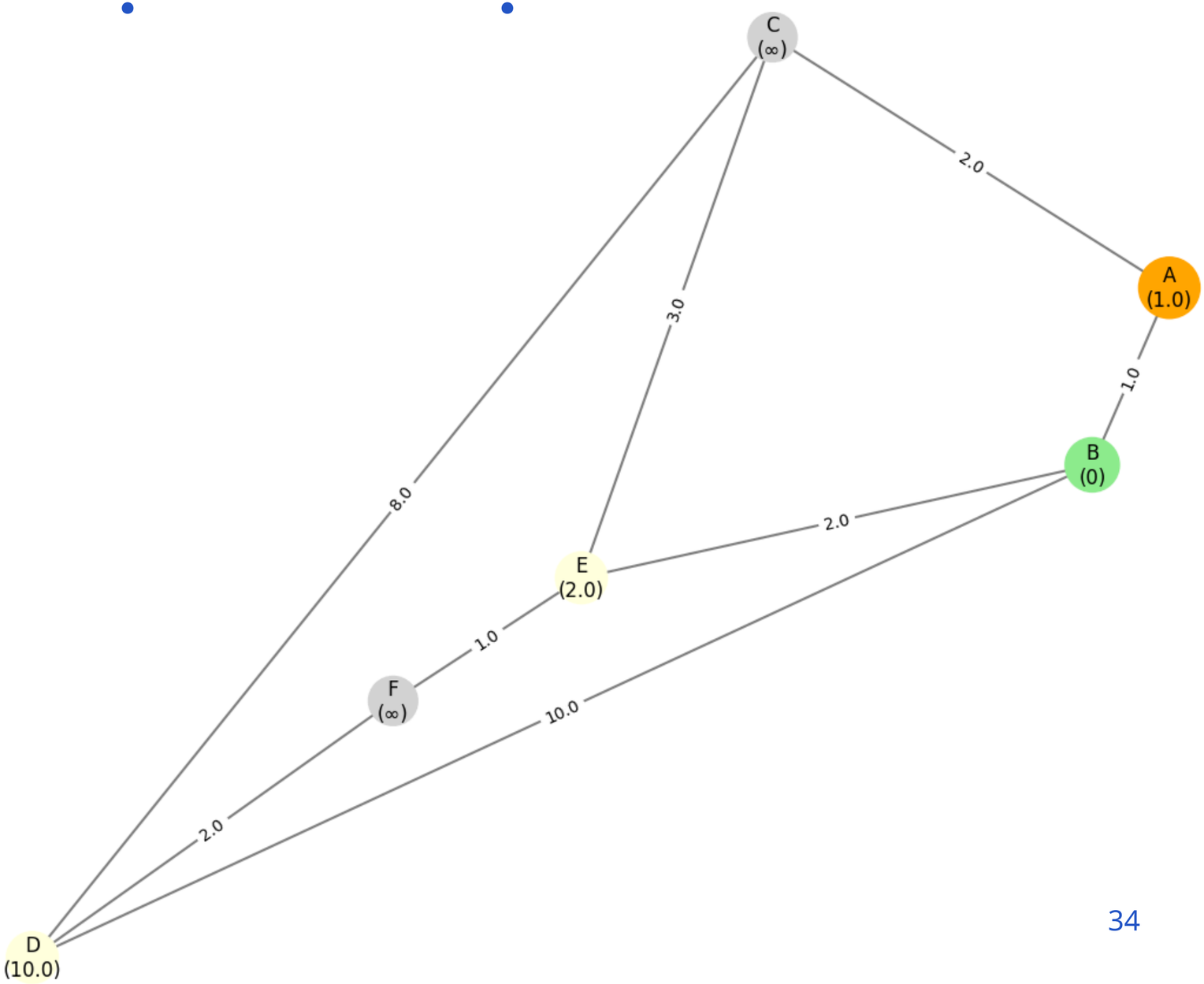
Đỉnh nguồn: B

Đỉnh đích: D



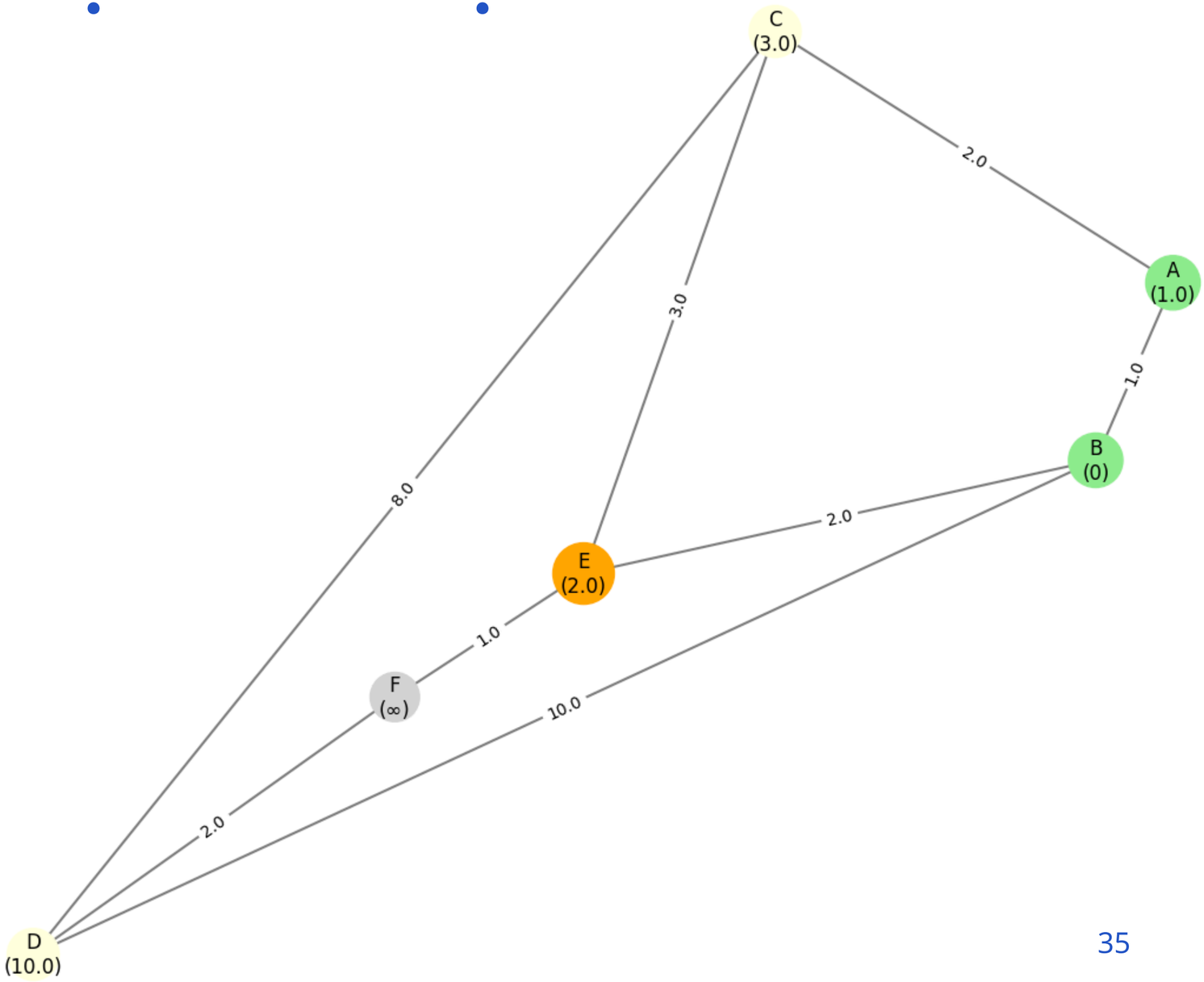
KẾT QUẢ THỰC NGHIỆM - TH1

- Từ đỉnh B, duyệt các đỉnh kề v.
- Cập nhật dist[v] và đẩy v vào hàng đợi ưu tiên.
- Lấy đỉnh A có khoảng cách nhỏ nhất từ hàng đợi.
- đánh dấu đỉnh B đã được duyệt.



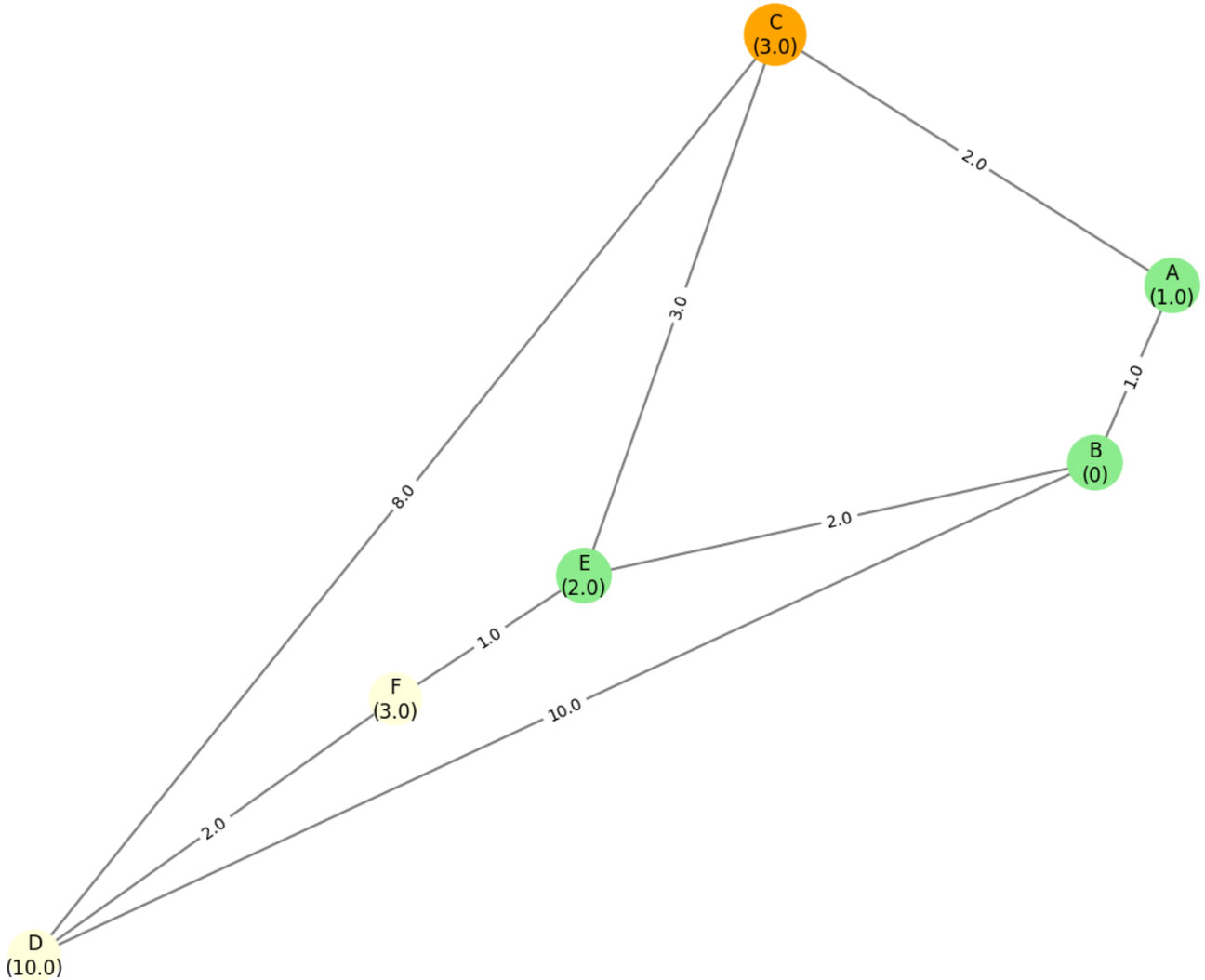
KẾT QUẢ THỰC NGHIỆM - TH1

- Từ đỉnh A, duyệt các đỉnh kề v.
- Cập nhật dist[v] và đẩy v vào hàng đợi ưu tiên.
- Lấy đỉnh E có khoảng cách nhỏ nhất từ hàng đợi.
- Đánh dấu đỉnh A đã được duyệt.



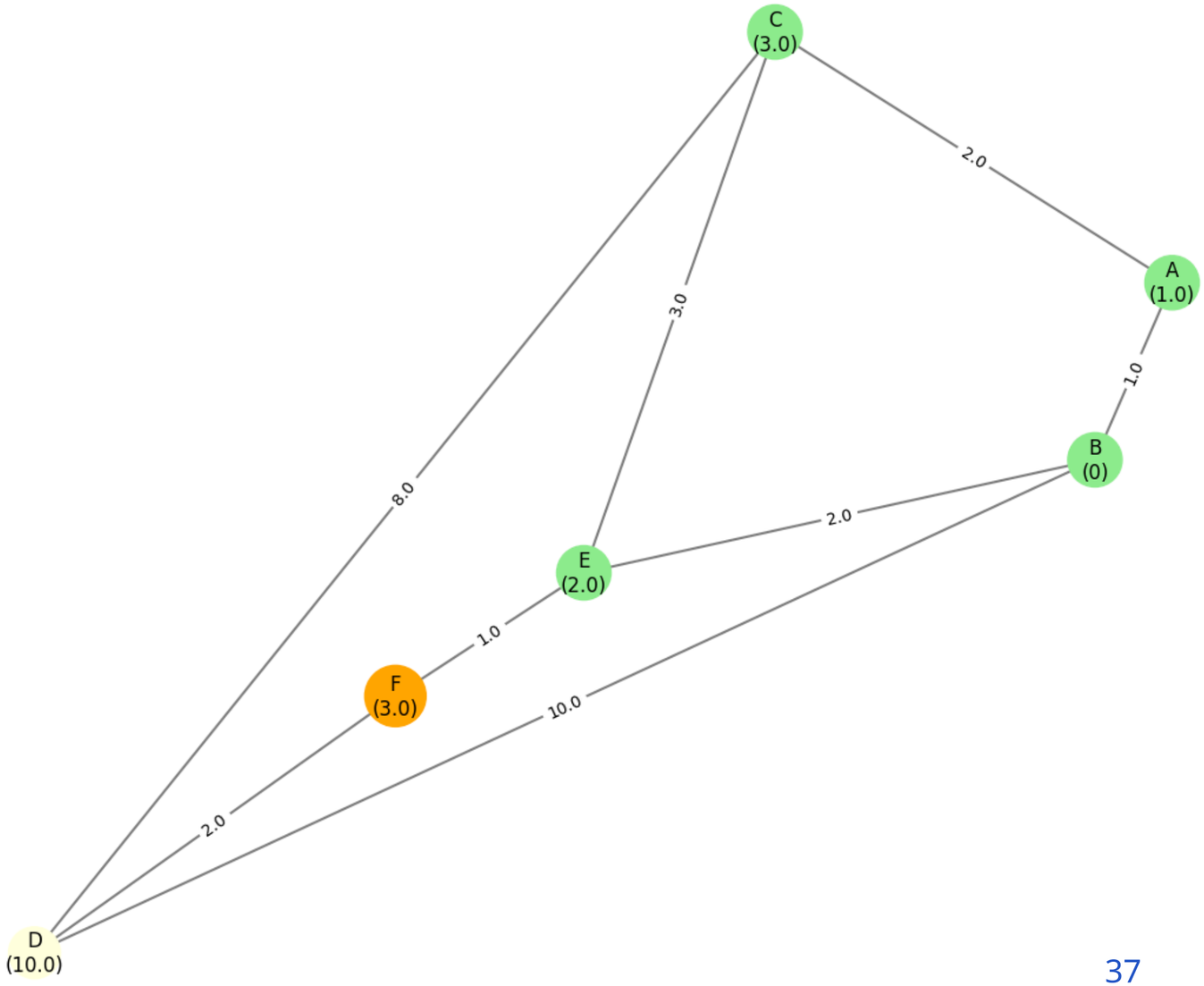
KẾT QUẢ THỰC NGHIỆM - TH1

- Từ đỉnh E, duyệt các đỉnh kề v.
- Cập nhật dist[v] và đẩy v vào hàng đợi ưu tiên.
- Lấy đỉnh C có khoảng cách nhỏ nhất từ hàng đợi.
- Đánh dấu đỉnh E đã được duyệt.



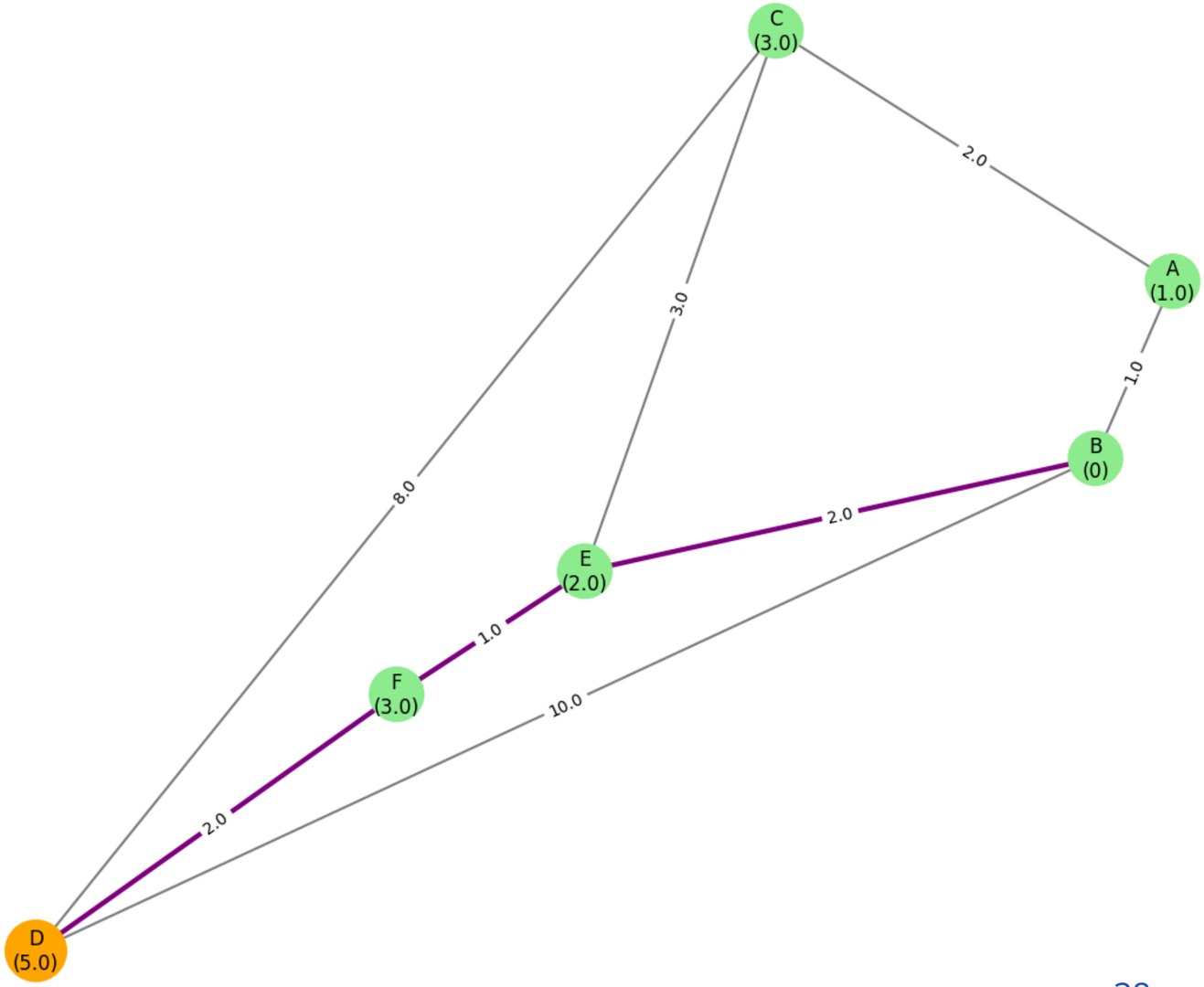
KẾT QUẢ THỰC NGHIỆM - TH1

- Từ đỉnh C, duyệt các đỉnh kề v.
- Cập nhật dist[v] và đẩy v vào hàng đợi ưu tiên.
- Lấy đỉnh F có khoảng cách nhỏ nhất từ hàng đợi.
- Đánh dấu đỉnh C đã được duyệt.



KẾT QUẢ THỰC NGHIỆM - TH1

- Từ đỉnh F, duyệt các đỉnh kề v.
- Cập nhật dist[v] và đẩy v vào hàng đợi ưu tiên.
- Lấy đỉnh D có khoảng cách nhỏ nhất từ hàng đợi
- Đánh dấu đỉnh F đã được duyệt.



Kết quả: Đường đi B → E → F → D ngắn nhất với tổng chi phí bằng 5

KẾT QUẢ THỰC NGHIỆM - TH2

CẤU TRÚC FILE CSV

A,B,1

A,C,2

B,D,4

B,E,2

C,E,3

C,D,3

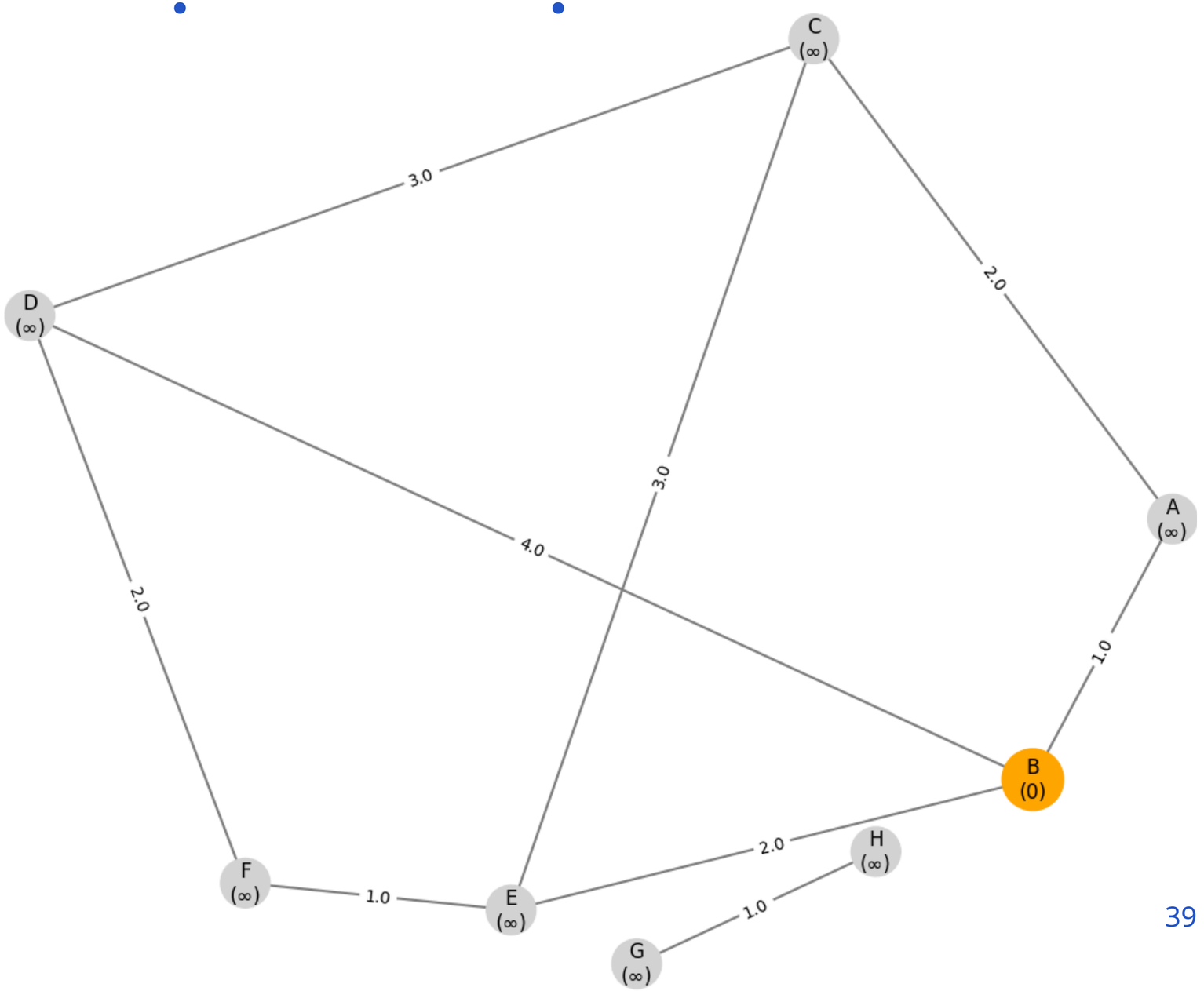
E,F,1

D,F,2

G,H,1

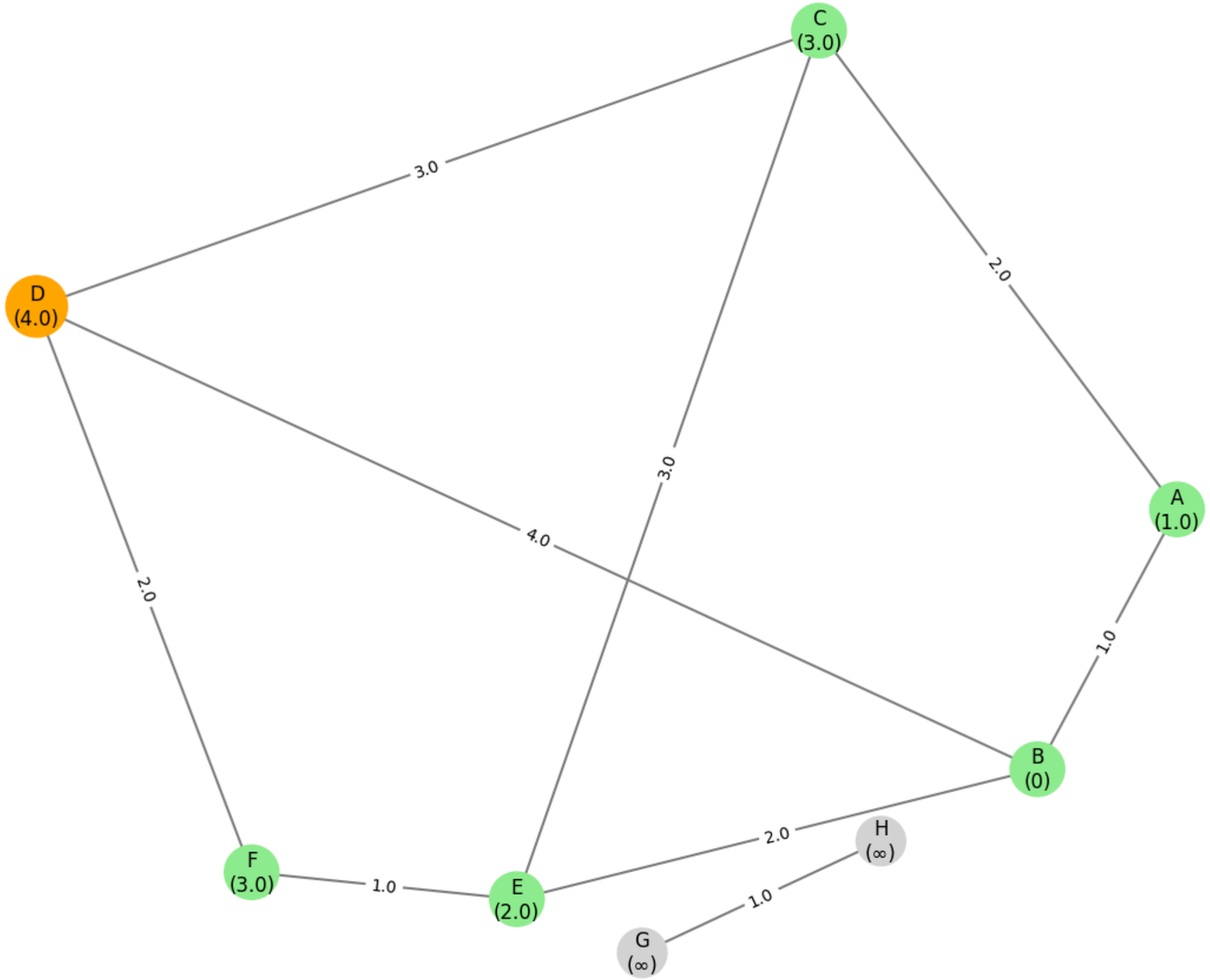
Đỉnh nguồn: B

Đỉnh đích: G



KẾT QUẢ THỰC NGHIỆM - TH2

- Đồ thị xuất phát từ đỉnh B đã được duyệt toàn bộ.
- Tuy nhiên, đồ thị chứa điểm bắt đầu (B) và đồ thị chứa điểm kết thúc (G) không liên thông.



Kết quả: Hai đỉnh B và G không có đường đi.

KẾT LUẬN

- Thuật toán Dijkstra là một thuật toán kinh điển dùng để tìm đường đi ngắn nhất từ đỉnh nguồn đến tất cả các đỉnh khác trong đồ thị có trọng số không âm.
- Khác với A*, Dijkstra không sử dụng hàm heuristic, nên phải duyệt qua toàn bộ không gian trạng thái trước khi xác định kết quả tối ưu.
- Do đó, thời gian tính toán tăng mạnh khi kích thước bản đồ lớn hoặc các điểm trong bản đồ được kết nối dày đặc.

Thanks for listening!